

3 FSDP Architecture

[**TODO:** Opening paragraph (strongest in the section). FSDP shards model state across GPUs and gathers the parameters of one unit only when that unit computes. Preview Sections 3.1 to 3.11.]

3.1 Notation and Definitions

[**TODO:** Short intro sentence to the notation.]

Symbol	Meaning
N	Number of GPUs
Ψ	Total number of model parameters
Ψ_u	Number of parameters in one FSDP unit
W	Parameters of one unit (flattened)
W_i	The i -th shard of W , owned by rank i
$g^{(i)}$	Local gradient computed on GPU i
g_i	The i -th shard of the averaged gradient g
K	Optimizer-state bytes per parameter

Table 1: Notation used in this section.

Definition 1 (Sharding). [**TODO:** Splitting a tensor into N disjoint pieces, where rank i owns piece i .]

Definition 2 (FSDP unit). [**TODO:** A group of layers whose parameters are flattened, sharded, gathered, and freed together.]

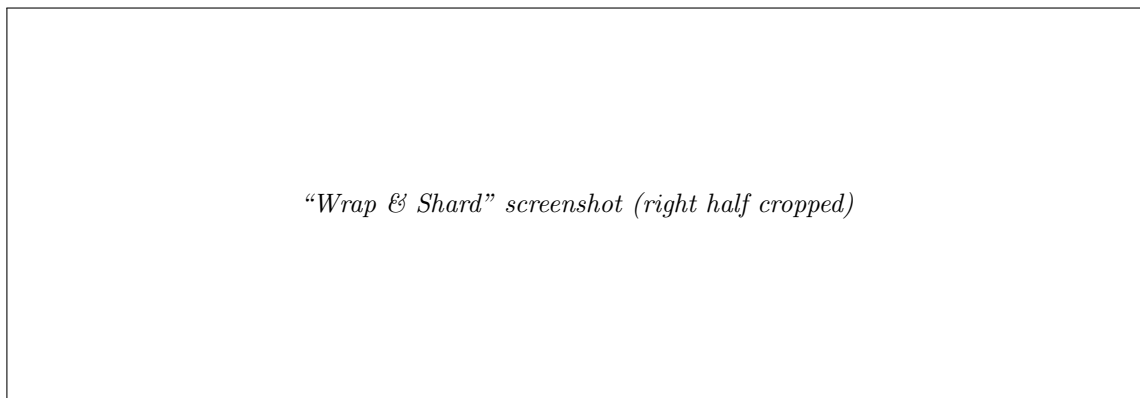


Figure 1: Wrapping a model into FSDP units. Layers 0 and 3 are not wrapped separately and therefore belong to the outer unit, Unit0. [**TODO:** finalise caption]

[**TODO:** Paragraph referring to Figure 1; point out that Layers 0 and 3 belong to the outer Unit0.]

3.2 Model State and Sharding

3.2.1 Components of Model State

[**TODO:** Parameters, gradients, and optimizer states. For Adam with mixed precision, the optimizer state is the fp32 master weights plus the first and second moments.]

3.2.2 Replication in DDP versus Sharding in FSDP

[**TODO:** Paragraph contrasting full replication (DDP) with sharding (FSDP); refer to Figure 2.]

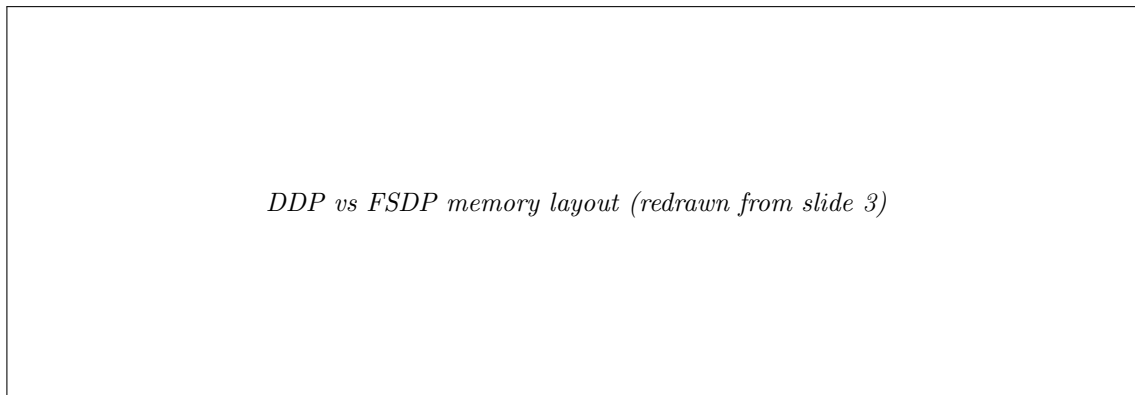


Figure 2: Model-state memory layout under DDP (every GPU holds a full replica) and FSDP (every GPU holds one shard). [**TODO:** finalise caption]

3.2.3 Flattening and Padding

[**TODO:** Explain FlatParameter: a unit's parameters are concatenated into one 1-D tensor, padded to a multiple of N , then split evenly. Refer to Figure 3 and Equation (1).]

$$\text{shard size} = \left\lceil \frac{\Psi_u}{N} \right\rceil \quad (1)$$

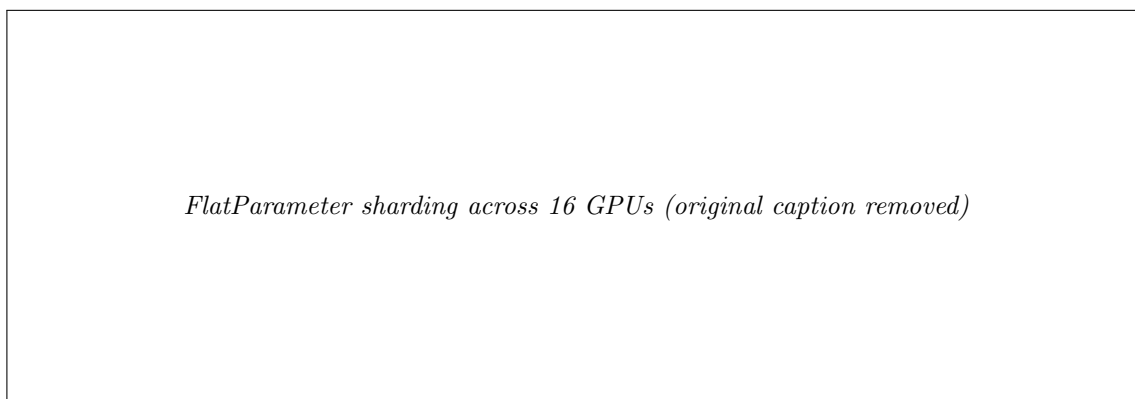


Figure 3: FlatParameter sharding across 16 GPUs. The symbol F in the figure corresponds to N in our notation. [**TODO:** finalise caption]

3.3 Collective Communication Primitives

3.3.1 All-Gather

[**TODO:** Shards become the full tensor on every rank.]

$$V_{\text{AG}} = \frac{N-1}{N} |W| \quad (2)$$

3.3.2 Reduce-Scatter

[**TODO:** Sum the tensors across ranks; each rank keeps one reduced shard. Stress that it is a single collective, not a reduce followed by a scatter.]

$$V_{\text{RS}} = \frac{N-1}{N} |G| \quad (3)$$

3.3.3 Relation to All-Reduce

[**TODO:** All-Reduce = Reduce-Scatter + All-Gather; refer to Figure 4. Mention the figure shows sums while FSDP averages.]

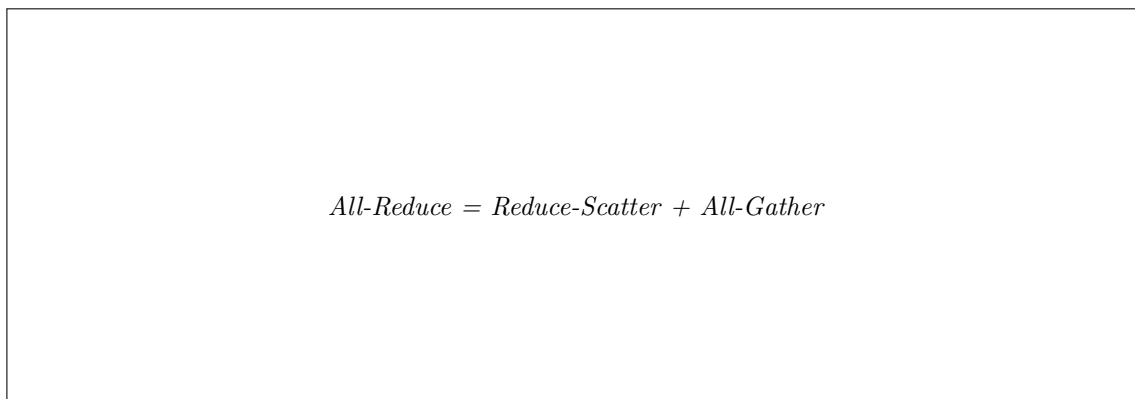

$$\text{All-Reduce} = \text{Reduce-Scatter} + \text{All-Gather}$$

Figure 4: All-Reduce decomposed into Reduce-Scatter followed by All-Gather. The figure shows sums; FSDP additionally divides by N to obtain the average. [**TODO:** finalise caption]

3.4 Forward Pass

3.4.1 Why a Layer Needs Full Parameters

[**TODO:** $y = Wx$ needs all of W , not just W_i .]

3.4.2 Gather, Compute, Reshard

[**TODO:** All-gather W , compute, then each rank frees the parameters it does not own. The full parameters are transient; the shard is persistent.]

3.5 Backward Pass and Gradient Synchronization

3.5.1 Re-gathering Parameters

[**TODO:** Parameters must be gathered again because they were resharded after the forward pass.]

3.5.2 Local Gradients

[**TODO:** Each GPU computes gradients on a different mini-batch, so the gradients differ across GPUs.]

3.5.3 Synchronization by Reduce-Scatter

[**TODO:** Paragraph introducing Equation (4) and the worked example in Table 2.]

$$g = \frac{1}{N} \sum_{i=1}^N g^{(i)}, \quad \text{rank } i \text{ keeps only } g_i \quad (4)$$

Table 2: Worked four-GPU example of gradient synchronization by Reduce-Scatter (from slide 17).

[**TODO:** fill in local gradients]

GPU	Local gradient $g^{(i)}$	Shard kept g_i
0	[TODO:]	1.5
1	[TODO:]	1.5
2	[TODO:]	2.0
3	[TODO:]	1.5
Averaged gradient $g = [1.5, 1.5, 2.0, 1.5]$		

3.6 Optimizer Step

[**TODO:** Each rank updates its own shard locally, because it holds the matching parameter, gradient, and optimizer-state shards. No communication is needed.]

3.7 The Complete Training Iteration

[**TODO:** Complete lead-in sentence ending in a colon, e.g.:] Algorithm 1 summarises one FSDP training iteration on rank i :

3.8 Memory Analysis

3.8.1 Persistent Memory

[**TODO:** Paragraph: fp16 parameters (2 bytes), fp16 gradients (2 bytes), and Adam optimizer state with $K = 12$ (fp32 master weights, first and second moments).]

$$M_{\text{DDP}} = (2 + 2 + K) \Psi = 16\Psi \text{ bytes} \quad (5)$$

Proposition 1. *Under full sharding across N GPUs, the persistent model-state memory per GPU is*

$$M_{\text{FSDP}} = \frac{16\Psi}{N} \text{ bytes.} \quad (6)$$

Proof. [**TODO:** Short derivation: each of the three components is split into N disjoint shards (Definition 1), so each rank stores $(2 + 2 + K)\Psi/N$ bytes.] $\square$

Algorithm 1 One FSDP training iteration on rank i

Require: Parameter shards $W_i^{(u)}$ and optimizer-state shards for every unit $u = 1, \dots, U$; local mini-batch $x^{(i)}$
 // Forward pass
1: **for** $u = 1$ **to** U **do**
2: $W^{(u)} \leftarrow \text{ALLGATHER}(W_i^{(u)})$
3: Compute forward of unit u with $W^{(u)}$
4: Free all of $W^{(u)}$ except $W_i^{(u)}$ ▷ reshard
5: **end for**
6: Compute loss $\mathcal{L}$
 // Backward pass
7: **for** $u = U$ **down to** 1 **do**
8: $W^{(u)} \leftarrow \text{ALLGATHER}(W_i^{(u)})$
9: Compute local gradient $g^{(i),u}$ of unit u
10: Free all of $W^{(u)}$ except $W_i^{(u)}$
11: $g_i^{(u)} \leftarrow \text{REDUCE_SCATTER}(g^{(i),u})/N$
12: Free $g^{(i),u}$
13: **end for**
 // Optimizer step (local, no communication)
14: **for** $u = 1$ **to** U **do**
15: Update $W_i^{(u)}$ using $g_i^{(u)}$ and its optimizer-state shard
16: **end for**

3.8.2 Peak Memory

[**TODO:** Explain the transient terms.]

$$M_{\text{peak}} \approx \frac{16\Psi}{N} + \underbrace{(2+2)\Psi_u}_{\text{gathered unit(s)}} + M_{\text{act}} \quad (7)$$

3.8.3 Worked Example

[**TODO:** A 7.5B-parameter model on 64 GPUs: DDP needs $16 \times 7.5 \times 10^9 \approx 120$ GB per GPU; full sharding needs $120/64 \approx 1.9$ GB per GPU [1].]

3.9 Communication Analysis and Overlap

3.9.1 Communication Volume

[**TODO:** DDP: one All-Reduce = RS + AG (Figure 4), about 2Ψ . FSDP: AG in forward, AG in backward, RS of gradients, about 3Ψ .]

$$V_{\text{DDP}} \approx 2\Psi, \quad V_{\text{FSDP}} \approx 3\Psi, \quad \frac{V_{\text{FSDP}}}{V_{\text{DDP}}} \approx 1.5 \quad (8)$$

3.9.2 Overlapping Communication with Computation

[**TODO:** Prefetch unit $i+1$ while unit i computes; refer to Figure 6.]



Figure 5: FSDP workflow. **[TODO: Relabel or explain “N layers / M layers”; explain or remove the CPU-offload labels.]**

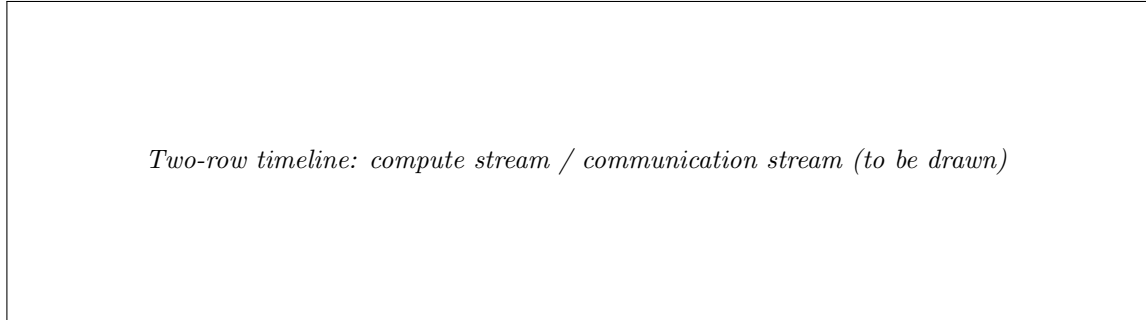


Figure 6: Overlapping communication with computation. While unit i computes on the compute stream, the All-Gather for unit $i+1$ runs on the communication stream. **[TODO: finalise caption]**

3.10 Sharding Strategies

[TODO: Paragraph covering FULL_SHARD, SHARD_GRAD_OP, NO_SHARD, and HYBRID_SHARD; refer to Table 3.]

3.11 Trade-offs: FSDP versus DDP

[TODO: Paragraph introducing Table 4 (extends slide 19).]

[TODO: Closing summary paragraph: FSDP trades about $1.5\times$ communication for roughly N -fold lower persistent memory, and overlap (Section 3.9.2) hides part of that extra cost.]

References

- [1] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. ZeRO: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.
- [2] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, A. Desmaison, C. Balioglu, P. Damania, B. Nguyen, G. Chauhan, Y. Hao, A. Math-

Table 3: FSDP sharding strategies. **[TODO: check wording]**

Strategy	Sharded state	Reshard after fwd?	Memory / communication effect
FULL_SHARD	Params, grads, optimizer state	Yes	Lowest memory; $\sim 3\Psi$ communication
SHARD_GRAD_OP	Grads, optimizer state	No	Params stay gathered between fwd and bwd; saves one All-Gather ($\sim 2\Psi$), higher memory
NO_SHARD	Nothing	–	Equivalent to DDP; 16Ψ per GPU, $\sim 2\Psi$ communication
HYBRID_SHARD	Full shard within a node, replicate across nodes	Yes	Memory reduced by the intra-node group size; slow inter-node links only carry gradient All-Reduce

(Optional) Hybrid sharding, from the VLDB paper

Figure 7: Hybrid sharding [2]. **[TODO: finalise caption or remove figure]**

ews, and S. Li. PyTorch FSDP: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 16(12):3848–3860, 2023.

Table 4: DDP versus FSDP (full sharding). **[TODO: check numbers]**

	DDP	FSDP
Model-state memory per GPU	16Ψ	$16\Psi/N$
How parameters are obtained	Full local replica	All-Gather per unit, just in time
Gradient collective	All-Reduce	Reduce-Scatter
Communication volume	$\sim 2\Psi$	$\sim 3\Psi$ ($1.5\times$)
Main bottleneck	Memory	Communication bandwidth